

LexAid Project Report

1. Concept Development

The idea of LexAid came from my study experience and a lot of conversations with law students and legal practitioners. During my time as a student I have met many people who study law or work in law firms. When I talked to them about their daily work, most of them said the same thing: there is no mobile app that is really made for legal people. They use normal tools like a to-do app, notes, or spreadsheets, but those tools do not understand the legal working context. For example, a case has many different deadlines, and if a lawyer misses one deadline, it can cause real problems for the client. Also, legal English is full of Latin words (like "habeas corpus" or "estoppel") and some normal words that have very different meanings in law, so law students and junior lawyers need to remember a lot of vocabulary. And when they need to calculate court fees or count working days for a compensation case, they have to search the rules again and again to demonstrate to clients as to why such calculation standards would apply. These are all small things, but they happen every day and they waste a lot of time.

So after talking with them in detail, I decided to build a mobile legal assistant. The basic idea is to put four kinds of work together in one app: case management, a to-do list, legal calculators, and a vocabulary learning system. But just putting them together is not enough. From the conversations I confirmed several features that legal people really need.

First, the to-do list should be linked with cases. Legal work is always case-based. When a lawyer opens a case, they want to see what still needs to be done for this case. So in LexAid, when you create a task, you can choose which case it belongs to, and later you can filter the tasks by case. This makes the workflow much clearer.

Second, after the user inputs a court date in the case page, the app shows a weather forecast for that date. I learned from lawyers that weather can actually affect court cases. Sometimes a case is postponed because of extreme weather, or a lawyer has trouble traveling to the court, or the other side's lawyer cannot arrive and that changes the situation. So it is useful to check the weather for the court date directly in the app anytime, without opening another weather app. This feature uses the Open-Meteo API. In the future I want to add strike alerts and other reminders for court dates too, because strikes can also delay hearings.

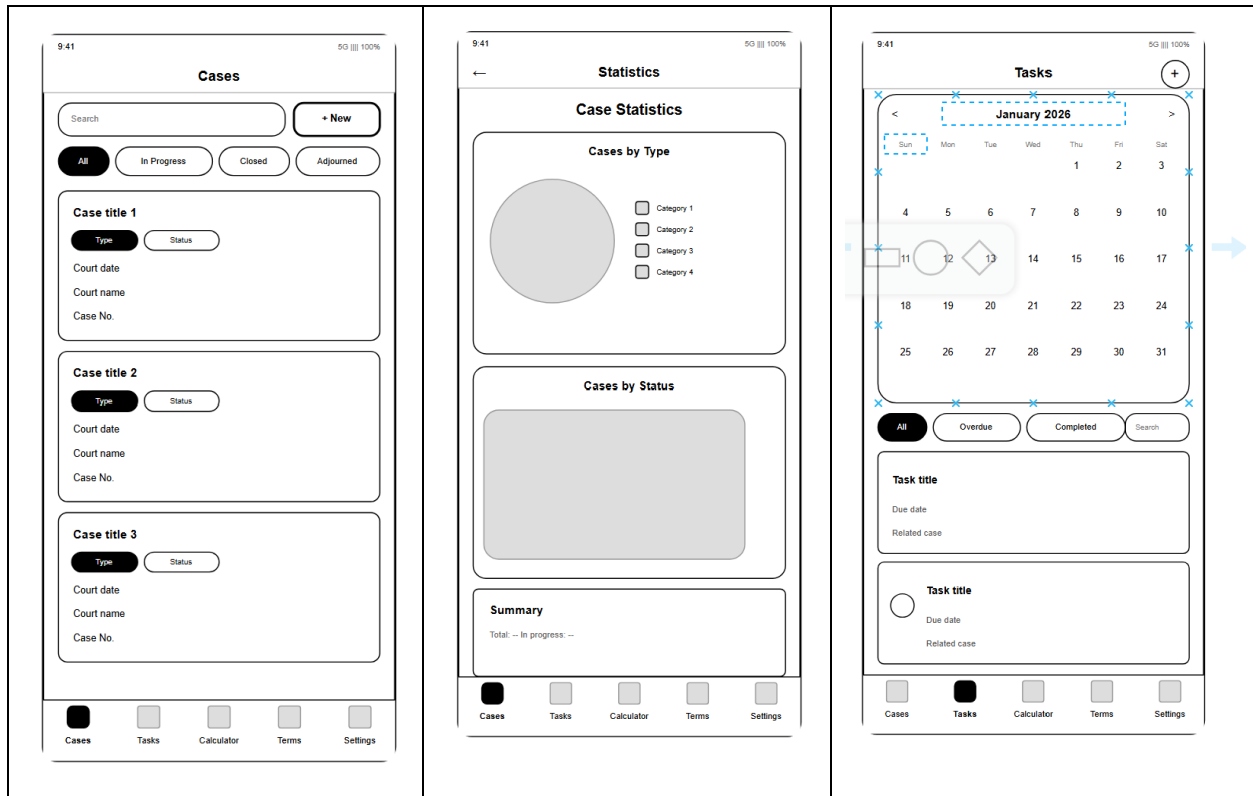
Third, legal English is very special. There are many Latin terms, and many words that do not mean the same as in daily English. The lawyers told me they meet new legal terms almost every day, not only Latin ones but also professional words from other industries. So the app has a flashcard system where the user can add any term they meet and review them in free time with spaced repetition. The terms in the app follow England and Wales law.

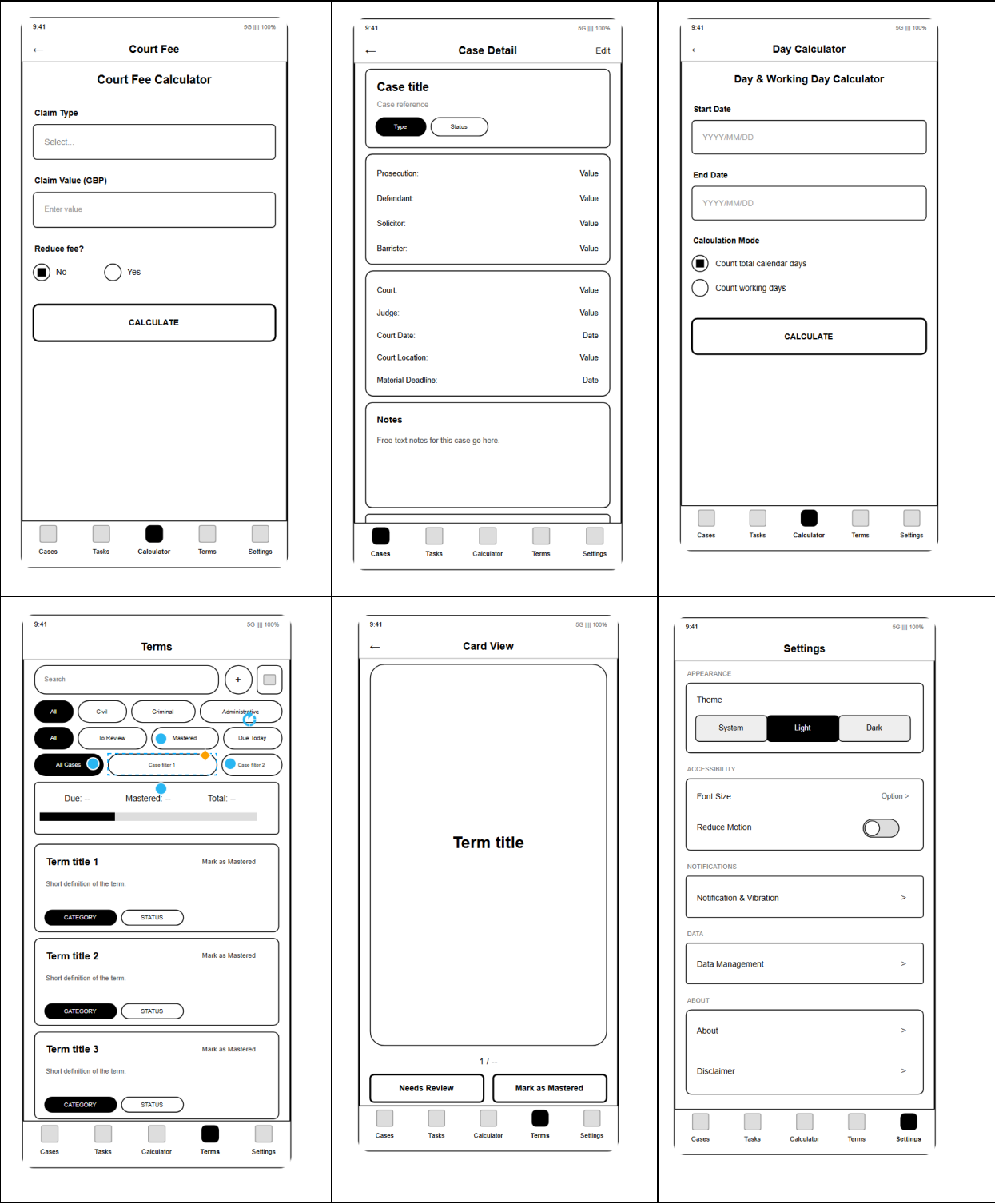
Fourth, about the calculators. Besides the normal court fee and solicitor fee calculators, I added a working day calculator. This is because in personal injury cases, employment law cases and compensation cases, the number of working days is often the key part of the calculation. The lawyers I talked to said they calculate this by hand or with a normal calendar, which is slow and

easy to make mistakes. LexAid counts working days between two dates automatically, and it knows the England and Wales bank holidays.

The value of LexAid is that it connects these four kinds of work together and keeps the data private. All the data is stored locally on the phone, because legal information is sensitive and the users do not want to upload it to some unknown server. As a course project, the app also clearly says in the About page that it is a teaching prototype and it does not provide formal legal advice.

2. Wireframing







3. Prototyping

After the wireframes, I made a prototype with Expo. The purpose of the prototype was to test the layout and the basic flow before writing all the real logic. I created a new Expo project and built the skeleton of the app: five bottom tabs (Cases, Tasks, Calculator, Terms and Settings), and each tab has its own stack for the sub pages. At this stage the pages were mostly placeholders with seed data, so people could click around and feel the structure.

I showed the prototype to some law students and asked them what they thought. The main feedback at this stage was about the tab names and the icons. For example, one person said he uses the calculators very often, so the Calculator tab should not be at the end. Another person said the difference between "Terms" and "Settings" was not clear from the icons, so I changed the icons and added a short description on the Terms page. The prototype stage helped me fix a lot of small design problems before they became expensive to change later.

4. User Feedback

I collected user feedback in three stages.

The first stage was after the wireframes and the prototype. I showed the screenshots to some law students and legal practitioners and asked them for their opinions. They gave me useful comments about the layout, for example which information should be on the case detail page and how the calendar should look.

The second stage was a Google Form questionnaire. After the first version of the app was working, I made a Google Form and sent it to more people. The form asked about their background, whether the purpose of the app was clear at first glance, the navigation, the calendar and reminders, the calculators, the flashcard function, the visual design, and which module they think is the most valuable.

Q1. What is your background? * ☐ Law student ☐ Solicitor / Legal practitioner ☐ Other Q2. I can easily understand the purpose of LexAid at first glance. * 1 2 3 4 5 Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree Q3. The bottom tab navigation (Cases / Tasks / Calculator / Terms / Settings) is intuitive. * 1 2 3 4 5 Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree Q4. I can quickly find and create a new legal case. * 1 2 3 4 5 Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree	Q5. The task calendar and deadline reminder functions are easy to use. * 1 2 3 4 5 Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree Q6. The legal calculator hub (court fee, statutory interest etc.) is useful for legal work. * 1 2 3 4 5 Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree Q7. The legal term flashcard review function helps with learning legal vocabulary. * 1 2 3 4 5 Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree Q8. The overall visual design, layout and spacing of the app look clean and professional. * 1 2 3 4 5 Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree
Q9. What module do you think is the most valuable feature? * ☐ Case management module ☐ Task & deadline calendar ☐ Legal calculator hub ☐ Legal terms flashcard library ☐ Case statistics & visualisation Q10. If this app was fully finished, would you use it? * ☐ Definitely yes ☐ Probably yes ☐ Not sure ☐ Probably no ☐ Definitely no	

From the answers I got some useful information, for example which layout looks better and which module people care about most. But I also found two problems with this method. First, the answers came in very slowly, so the feedback was not timely. Second, many people filled in the first form but did not want to fill in the third or fourth one. Also, most people left the open questions empty and only wanted to do the multiple choice ones. So this fixed format was not really good for collecting useful feedback.

So for the third stage, I built a WeChat group and invited some people I know well in the legal field. Every time I finished a module, I ran it on a real device (Expo Go) and showed it to them in the group. Instead of typing, they replied with voice messages, because typing takes much more time for them. Then I transcribed the voice messages and made changes based on them. This way the feedback was much faster and more natural. The whole development became a small-step iteration: build a module, show it, collect feedback, fix it, and test it again.

Here are some representative feedback rounds from the group:

- (1) The bottom tabs sometimes did not respond when clicked. I found the reason was that a `Pressable` component was nested inside the `tabBarIcon`, and it took over the touch events. I changed it to a separate `TabIcon` component and removed the inner `Pressable`, and added a scale animation when focused. The problem was solved.
- (2) The Appearance section on the Settings page overflowed, and the dark mode buttons went outside the container. I changed it to one row of "icon + label" and one row of three equal-width buttons below. It looks normal in dark mode now.
- (3) The search on the Tasks and Terms pages crashed. The problem was that the seed data marker keys were read as business data, and `JSON.parse` got a boolean value. I fixed it by excluding the marker keys when reading and adding a type check.

The continuous feedback also set two hard rules for the project: the interface and all the content must be in English, and all the legal concepts must follow England and Wales law. I followed these two rules in every module of the development.

5. Development

The app is built with React Native using Expo (SDK 54). All the code is JavaScript. The project structure is: `App.js` as the root, `navigation/` for the React Navigation setup, `screens/` for the page components, `utils/` for the pure business logic, `contexts/` for the global settings, and `data/` for the seed data.

I started with the data layer. The app stores everything in `AsyncStorage` on the phone, and on first launch it seeds some sample cases, tasks and terms so the app is not empty. A key thing I learned here is to use marker keys (like `@case_seed_done`) to remember that the seed has been imported, and the reading functions must skip these keys, otherwise `JSON.parse` crashes on a boolean value. This bug actually appeared later in the search feature and cost me some time to fix.

Then I built the navigation. The app has five bottom tabs, and each tab has its own stack so the sub pages can hide the bottom tab bar. I also made a custom tab icon component with a scale animation, and it respects the reduce motion setting from the global settings.

The Cases module was the first one I finished. It has a case list with search and status filter, a detail page with all the case fields and an evidence list, an edit page where the user can add or delete evidence and take photos with the camera (expo-image-picker and the new expo-file-system API), and a statistics page that shows charts of the case status using react-native-chart-kit. One interesting bug happened here: the chart color function parsed the primary color as a hex string, but the color was actually "rgb(0, 122, 255)", so it produced NaN values and the charts broke. I fixed it by parsing both formats.

The Tasks module has a calendar view (react-native-calendars), a task list with search and filters, and an edit page where the user can link a task to a case, set a due date, and choose whether to get a notification reminder. The calendar marks overdue days in red. The tasks also sync with the global notification switch in Settings. I had a crash on Android here: the date time picker with mode "datetime" is not supported on Android, and the library crashes when the picker closes, so I changed it to use "date" mode on Android and "datetime" on iOS.

The Calculator module has four calculators: court fee, solicitor fee, statutory interest, and day calculator. The legal rules are stored in data files, and the calculation logic is in utils so it can be unit tested. The day calculator counts calendar days or working days between two dates, and it uses the England and Wales bank holiday list (2023 and 2024), which I collected from [GOV.UK](https://www.gov.uk/bank-holidays). There is also a legal basis page that explains which rules each calculator uses.

The Terms module is a spaced repetition flashcard system. The user can add legal terms, link a term to a case, and review the cards by swiping. I wrote the scheduling logic myself in utils: "good" doubles the interval (capped at 30 days) and "again" resets it to 1 day. The review screen uses react-native-deck-swiper for the card swipe animation.

The Settings module controls the theme (light, dark or system), the font scale, reduce motion, vibration, and the notification switch. The settings are saved in AsyncStorage so they survive a restart. There are also pages for data management, accessibility settings, the disclaimer, and an About page.

The last feature I added was the weather API. On the case detail page, the app geocodes the court location, gets a 16 day forecast from Open-Meteo, and picks the day of the court date to show the weather and a travel risk note (low / moderate / high). If there is no internet, the page shows an error message with a retry button instead of crashing. This feature covers the cloud data topic from the module.

The hardest parts of the development were the Android specific bugs, because they only show up on a real device and not on the iOS simulator. I also learned that writing the logic in utils makes the screens much simpler and easier to test.

6. Unit Testing

For testing I used Jest with the jest-expo preset. The idea was to put all the business logic in utils/ as pure functions, so they can be tested without rendering any React component. This is also why the logic is separated from the UI.

There are six test files. The fee calculations tests cover the court fee bands, the validation errors, the solicitor fee ranges and the statutory interest. The term scheduler tests cover the spaced

repetition intervals, the 30 day cap, the due date calculation and the stats. The task scheduler tests cover the overdue detection, the filters, and the calendar marked dates. The holidays tests check the bank holiday dates for 2023 and 2024, the weekend adjustment, and the working day counting. The review flow test is an integration style test that simulates a user going through the spaced repetition deck. The weather tests cover the location cleaning, the forecast day lookup, the risk rules and the API request.

In total there are 74 tests and they all pass. Writing the tests also helped me find some edge cases, for example the interval logic when a term is reviewed for the first time (interval 0 should go to 1 day, not stay 0). And every time I added a new feature, like the weather API, I could run the whole test suite to make sure I did not break anything old.

```
Problems  Output  Debug Console  Terminal  Ports
● PS C:\Users\atlas\Desktop\Final> npm test

> lexaid@1.0.0 test
> jest

PASS  __tests__/reviewFlow.test.js (18.508 s)
PASS  __tests__/feeCalculations.test.js (18.394 s)
PASS  __tests__/holidays.test.js (18.712 s)
PASS  __tests__/taskScheduler.test.js (18.438 s)
PASS  __tests__/weather.test.js (18.457 s)
PASS  __tests__/termScheduler.test.js (18.444 s)

Test Suites: 6 passed, 6 total
Tests:       74 passed, 74 total
Snapshots:   0 total
Time:        26.316 s
Ran all test suites.
PS C:\Users\atlas\Desktop\Final>
```

7. Evaluation

Looking back at the goals in the concept development, most of them are achieved. The to-do list can be linked with case. The weather forecast for the court date works with the Open-Meteo API and gives a travel risk note. The flashcard system with spaced repetition works and the user can add their own terms. The calculators, including the working day calculator, are all functional. The data is stored locally, so the privacy requirement is also met.

From the user feedback, the WeChat group method worked much better than the questionnaire. The five feedback rounds I described all fixed real problems that I could not find by myself, especially the Android bugs. I think this shows that small-step iteration with real users is very effective for a mobile app project.

However, there are still some limitations. The weather forecast only covers 16 days, so if the court date is too far away, there is no forecast for it. The strike alerts that I mentioned in the concept are not implemented yet, and they would be a useful addition. The dark mode still has some hard coded colors that are not perfect in some pages. The app has no account system, so the data can only be on one device, and the weather feature needs an internet connection.

For the future, I would like to add the strike and hearing alerts, a better color system for dark mode, and maybe an optional cloud sync for users who want it.

For me personally, this project taught me a lot. I learned how to structure a React Native app, how to use navigation, hooks and storage, how to call an API and handle errors, how to test business logic with Jest, and most importantly, how to collect feedback from real users and turn it into changes. The whole development process, from concept to wireframing, prototyping, feedback, development and testing, gave me a much better understanding of how a mobile app is really made.